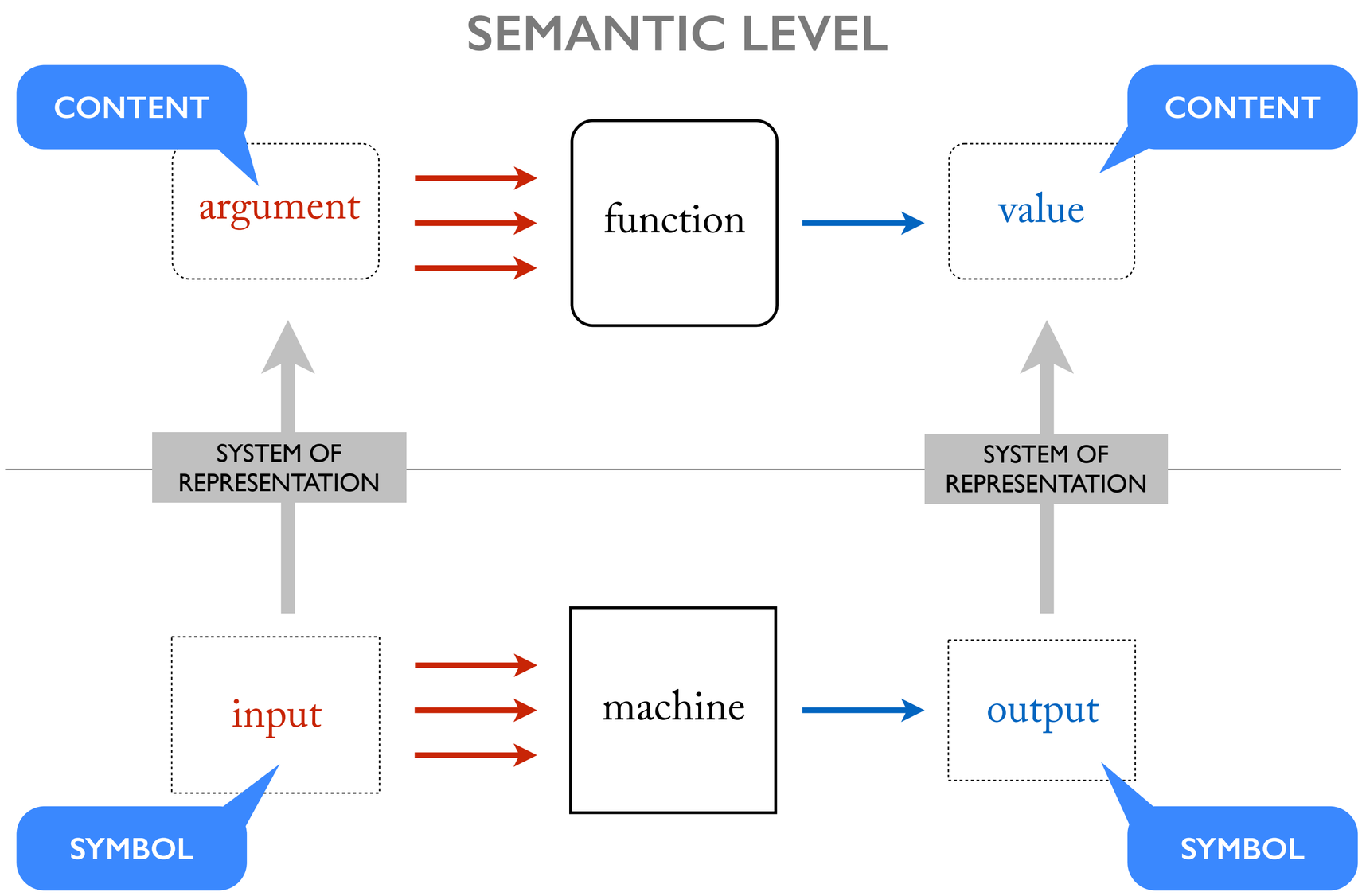
5. Systems of Representation

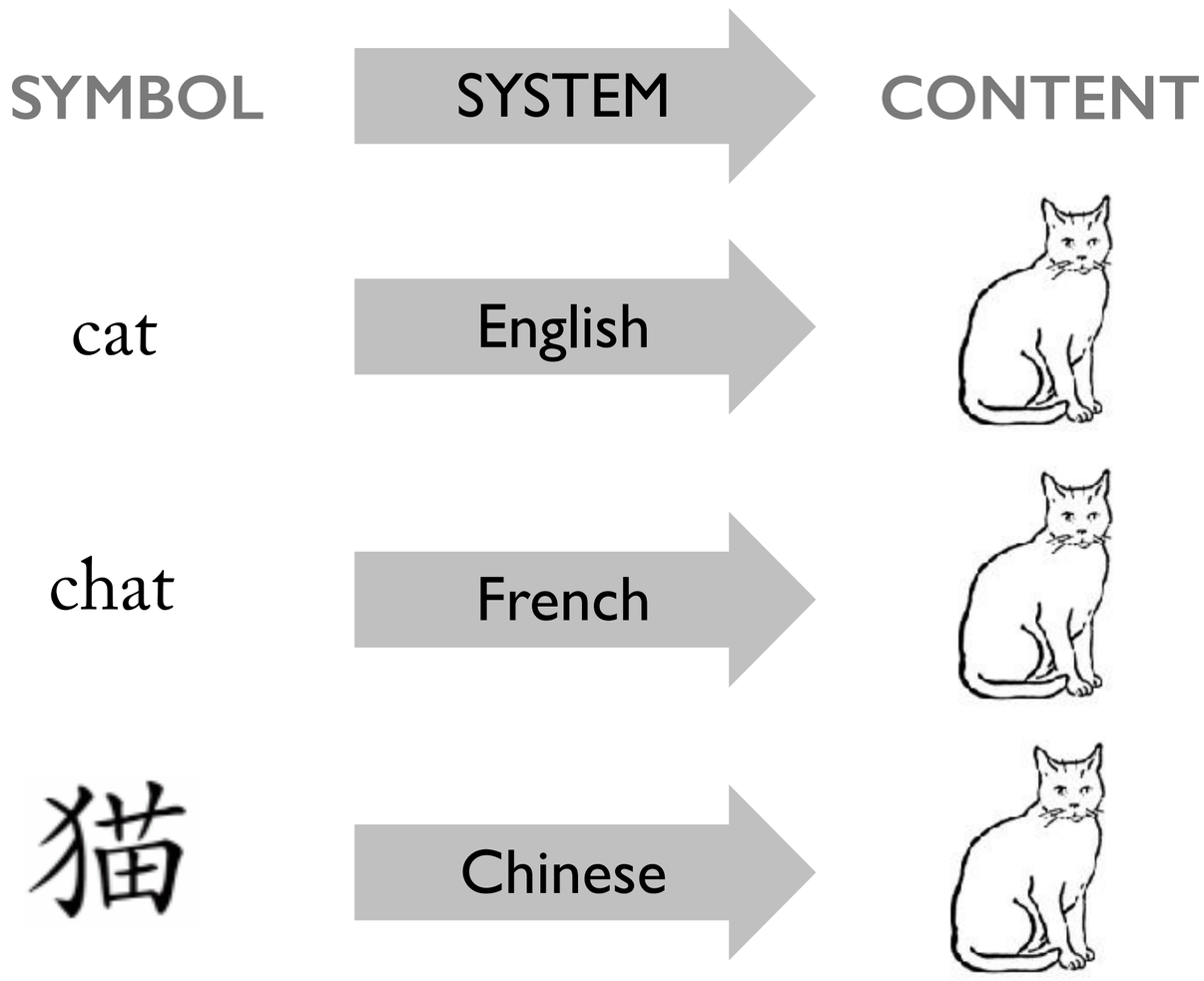
The Role of Representational Systems in Computation

Systems of representation associate symbols with contents. In computation, they map inputs to arguments and outputs to values. They are the glue that connects the physical and semantic levels.



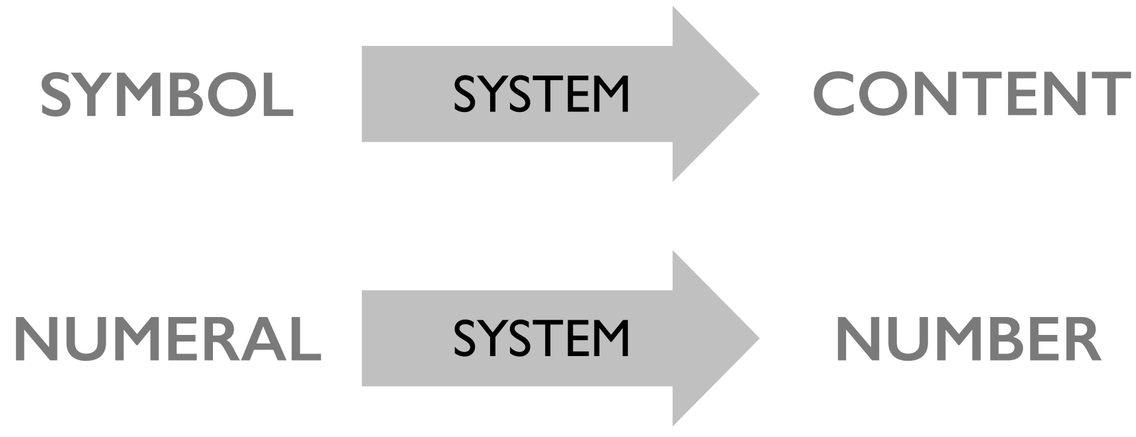
Symbol, Content, System

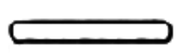
Symbols express contents only relative to a system. The same content can be expressed by different symbols in different systems. Here is an illustration for the case of words in a language.

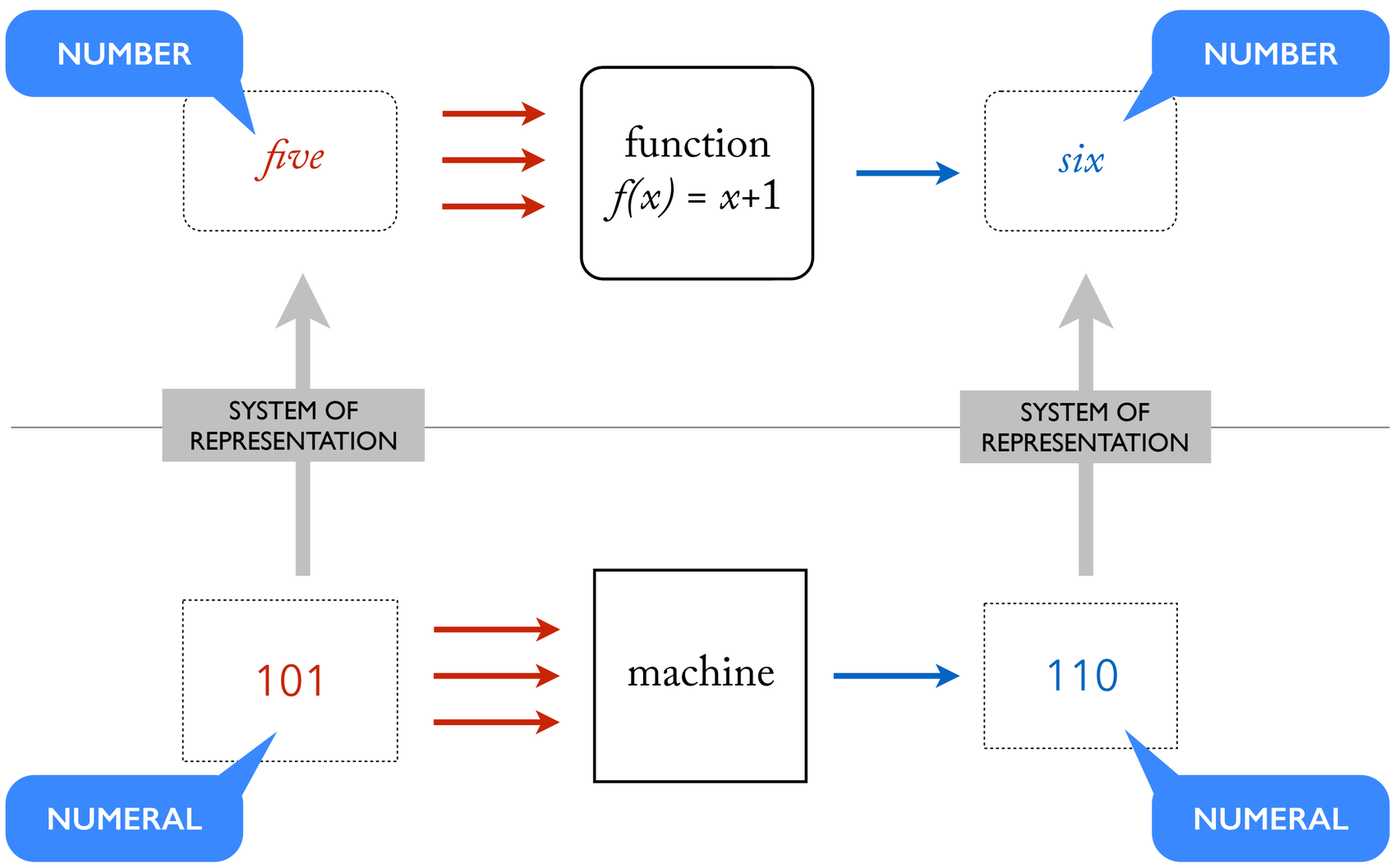


Numeral Systems

Numeral systems are systems of representation specifically for numbers. In these systems, the symbols are called **numerals**, and the contents are **numbers**. Numerals are types of physical symbols, while numbers are abstract quantities.



Different numeral systems use…	Roman numerals	Indo-Arabic numerals	Mayan numerals	The numbers represented
		0		*zero*
	I	1		*one*
	II	2		*two*
	III	3		*three*
	IV	4		*four*
	V	5		*five*



Anatomy of a Numeral

A **complex** numeral is made up of a string of **atomic** numerals. Each atomic numeral has a **position** within the complex numeral.

In the **Indo-Arabic numeral** system we all use, the atomic numerals are the numerals 0–9, and all other numerals are constructed from these.

3rd position	2nd position	1st position	
3	8	4	complex numeral
	atomic numeral	atomic numeral	

Atomic symbols in the **Indo-Arabic** system

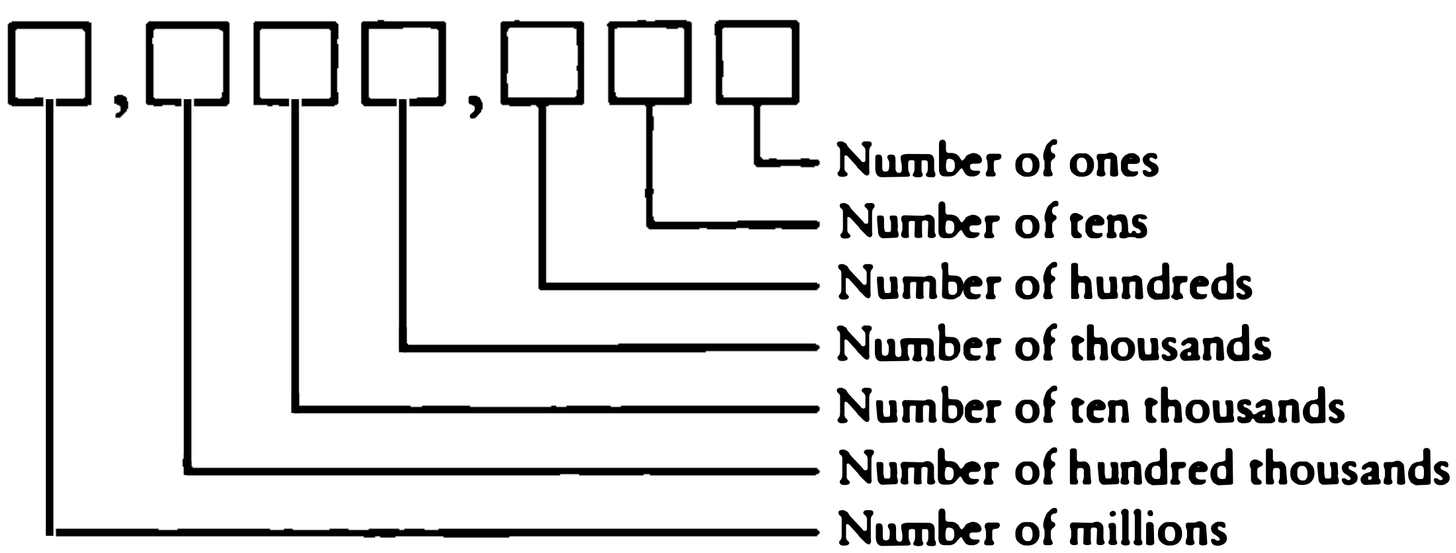
0 1 2 3 4 5 6 7 8 9

Mechanics of a Numeral System

A **positional** numeral system is defined by

(i) the **numeric values** it assigns to each **atomic numeral** (starting from zero);
(ii) the **numerical weight** of the **position rank** (starting from zero),
where the number of symbols in (i) = the weight in (ii).

In the Indo-Arabic system, the **weight** of the position rank is 10. That means that the weight of each position goes up by a power of 10. (The first position is considered position 0, so its weight is $10^0=1$.)



The Indo-Arabic numeral system is positional, but the Roman system is not. Both employ complex symbols to express large numbers, but they rely on different forms of representational system.

one hundred and one	101	CI
	Indo-Arabic numerals	Roman numerals

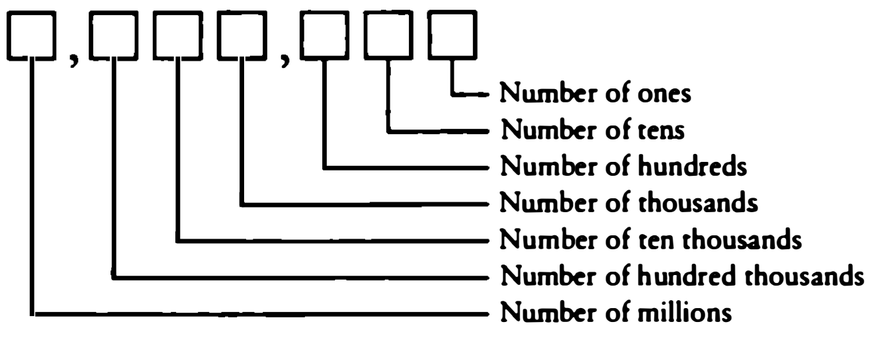
Note that the numeral 1 appears twice in the Indo-Arabic numeral, because it plays different roles in the different positions. In the Roman numeral, which is **additive** rather than positional, the numeral I only appears once, added to the value of C.

Base

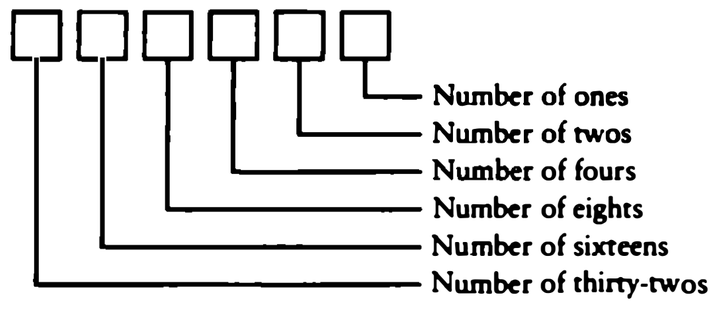
The **base** of a positional system corresponds to the number of atomic symbols. In the Indo-Arabic system, there are ten symbols, 0–9, so this system is called **base 10**. In the Binary system, there are only two symbols (0,1), so this system is **base 2**.

The base determines the weight of the positional rank. So in base 10, positions increase in weight by powers of 10, while in base 2, they increase by powers of 2.

Base 10



Base 2 (Binary)



Base 10	Base 7	Base 2 (Binary)	Base 1 (Tally)
1	1	1	1
2	2	10	11
3	3	11	111
4	4	100	1111
5	5	101	11111
6	6	110	111111
7	10	111	1111111
8	...	1000	11111111
9	15	1001	111111111
10	16	1010	1111111111
...	20	1011	11111111111
98	...	1100	111111111111
99	66	1101	1111111111111
100	100	1110	11111111111111

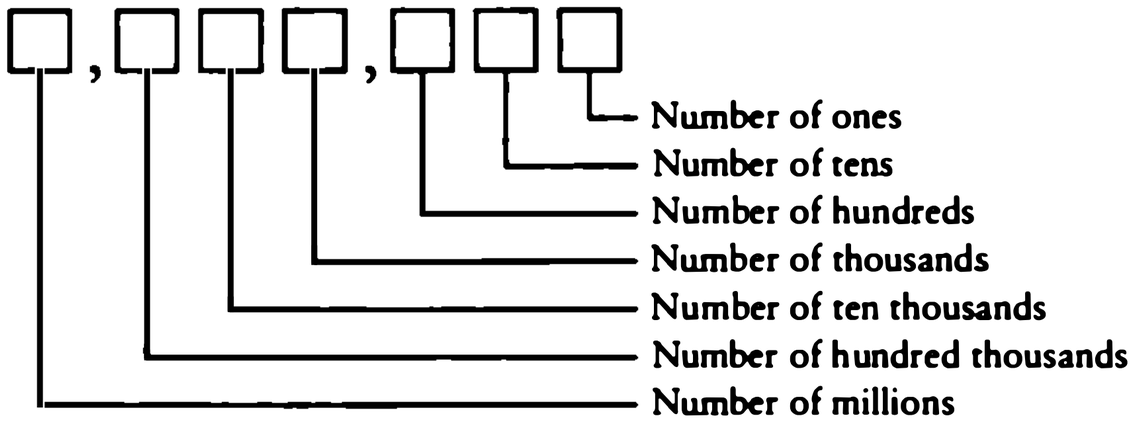
As you can see, Tally has the virtue of being very simple, but the vice of producing very large numerals— inefficient for creatures made of finite physical components.

Binary is of special significance in computer design because, like Tally, it only involves two possible digits, 1 and 0. But unlike Tally, the size of numerals increases non-linearly (~ logarithmically) as the number represented gets bigger. Today, binary is the representation system most used at the mechanical foundations of computing.

Calculating a Numeral's Value

Decimal System

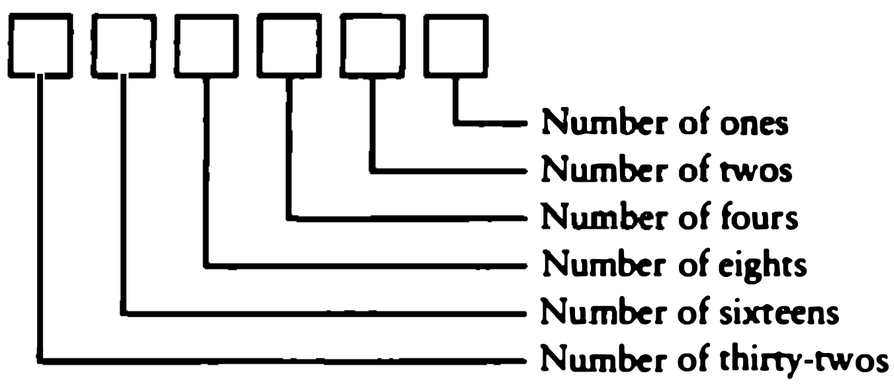
ten atomic symbols, *base* = 10



$$
\begin{aligned}
111_{D} &= 1_{D} \times 100 + \\
&\quad\ 1_{D} \times 10 + \\
&\quad\ 1_{D} \times 1 \\
&= 1 \times 100 + \\
&\quad\ 1 \times 10 + \\
&\quad\ 1 \times 1 \\
&= 100 + 10 + 1 \\
&= 111
\end{aligned}
$$

Binary System

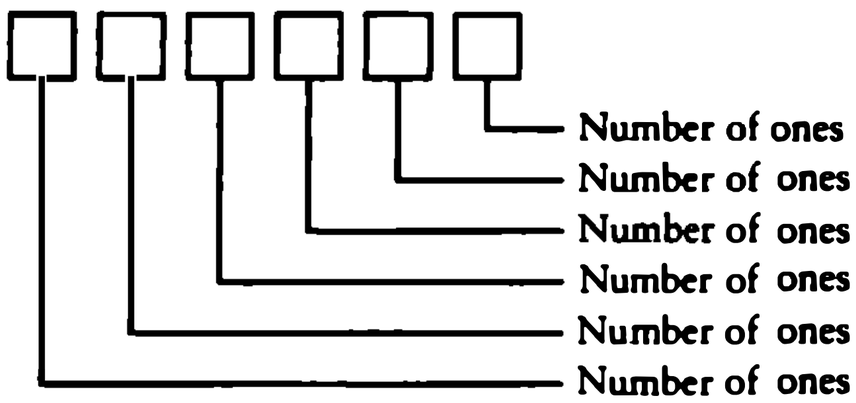
two atomic symbols, *base = 2*



$$
\begin{aligned}
111_{B} &= 1_{B} \times 4 + \\
&\quad\ 1_{B} \times 2 + \\
&\quad\ 1_{B} \times 1 \\
&= 1 \times 4 + \\
&\quad\ 1 \times 2 + \\
&\quad\ 1 \times 1 \\
&= 4 + 2 + 1 \\
&= 7
\end{aligned}
$$

Tally System

one atomic symbol, *base = 1*

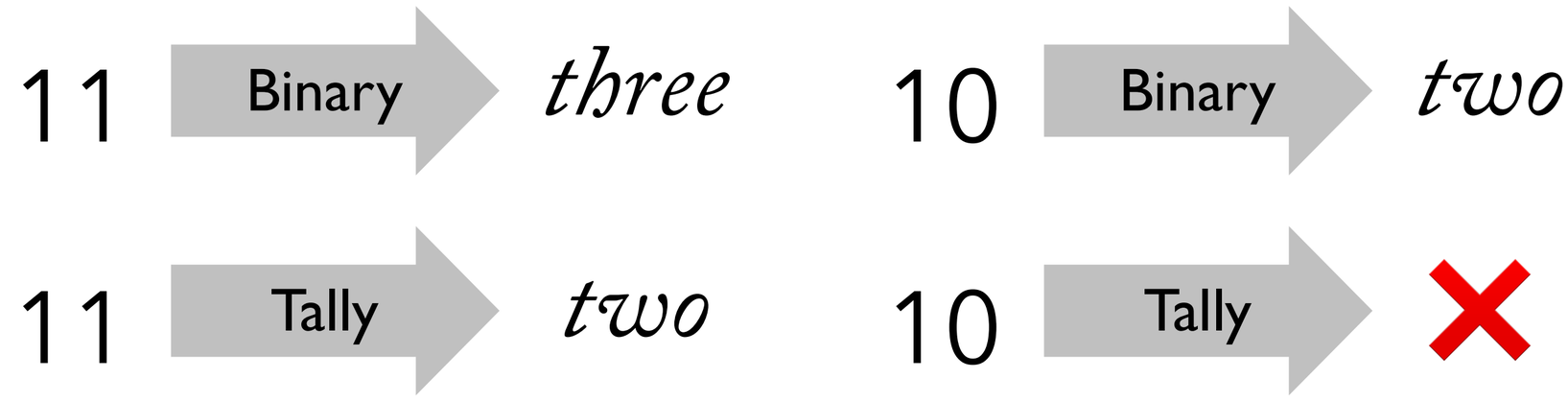


$$
\begin{aligned}
111_{T} &= 1_{T} \times 1 + \\
&\quad\ 1_{T} \times 1 + \\
&\quad\ 1_{T} \times 1 \\
&= 1 \times 1 + \\
&\quad\ 1 \times 1 + \\
&\quad\ 1 \times 1 \\
&= 1 + 1 + 1 \\
&= 3
\end{aligned}
$$

Binary and Tally as Representational Systems

In this course, the most important numeral systems will be **tally (T)** and **binary (B).** This is because you can express all numbers in these systems using only two symbols, 0 and 1.

Because these are two different ways of interpreting numerals, the same numerals can mean different things in the two systems.



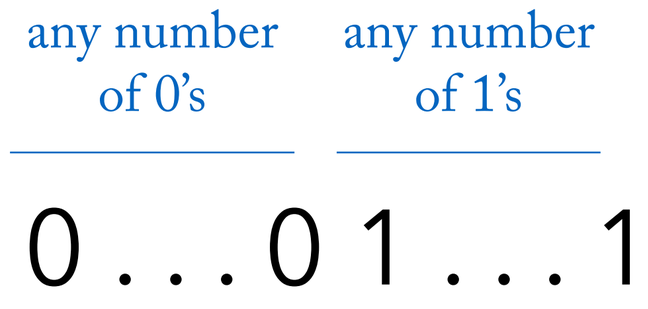
The **syntax** of a representational system is a set of rules for constructing well-formed complex numerals in that system. When designing a machine to compute a function in tally or binary, we'll always assume that inputs are well formed.

Tally Syntax

Tally systems in common usage take a variety of forms.



Here, a **well-formed tally numeral** is any string of bits, where the first n (≥ 0) bits from the right are 1's, and they are followed to the left by any number of 0's.



So the following are well formed:

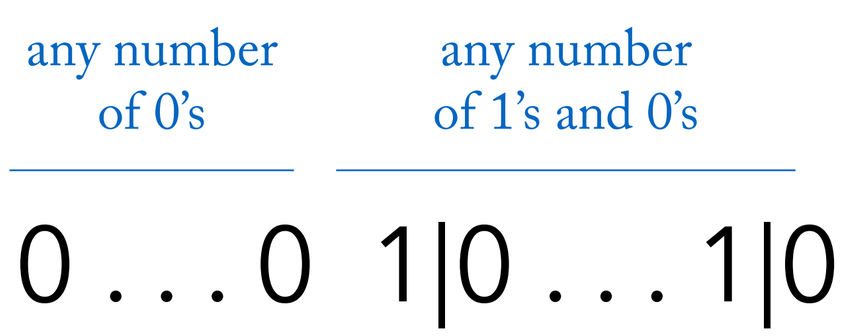
000
111
0011

But these ones are not.

1000
1101
1110

Binary Syntax

A **well-formed binary numeral** is any string of bits, where the first n (≥ 0) bits from the right are either 0's or 1's, and they are followed to the left by any number of 0's.



In binary, any sequence of 1's and 0's will be well formed.